DOCUMENTO TÉCNICO

Arquitectura del Sistema de Mercado Comportamental

(SMCTA)

Arquitectura Hexagonal · DDD · Diseño Conceptual del Producto White-Label

Julio 2026

1. Introducción y Alcance del Documento

Este documento describe el diseño técnico y arquitectónico del Sistema de Mercado Comportamental y Trading de Activos (SMCTA), a nivel conceptual y estructural. El objetivo es establecer los principios de diseño, los contextos de dominio, el modelo de datos y la superficie de integración (API) que deberá respetar cualquier implementación concreta del producto, sin atarse a una sintaxis de programación específica ni a decisiones de bajo nivel que corresponden a la fase de construcción.

El documento está dirigido a arquitectos de software, líderes técnicos y equipos de ingeniería que deban evaluar, extender o implementar el sistema, así como a stakeholders técnicos del negocio que necesiten comprender cómo la flexibilidad comercial descrita en el Documento de Visión se traduce en decisiones de arquitectura concretas.

2. Principios Arquitectónicos Rectores

La arquitectura de SMCTA se apoya en cuatro principios complementarios, elegidos específicamente para sostener el requisito comercial de ser un producto configurable y agnóstico al catálogo del cliente final.

2.1 Arquitectura Hexagonal (Ports & Adapters)

El sistema separa de forma estricta el núcleo de reglas de negocio (dominio) de todo aquello que es infraestructura: bases de datos, colas de mensajería, proveedores de pago, redes blockchain o protocolos de transporte (HTTP, WebSocket). El núcleo se comunica con el exterior únicamente a través de contratos llamados puertos; cada tecnología concreta se conecta a esos puertos mediante un adaptador.

La consecuencia práctica de este principio es que un cliente que adquiere el software puede sustituir su base de datos, cambiar su pasarela de pagos o incorporar una capa de blockchain sin que el equipo de desarrollo deba tocar la lógica central de cupones o del motor de precios. Sólo se construye un adaptador nuevo.

2.2 Domain-Driven Design (DDD) — Enfoque Táctico

El dominio se organiza en contextos acotados (Bounded Contexts), cada uno con su propio lenguaje, sus propias reglas y sus propios límites de responsabilidad. Esto evita que un cambio en, por ejemplo, la forma de calcular comisiones, tenga efectos colaterales imprevistos sobre la gestión del inventario o sobre la emisión de cupones.

2.3 Patrón Strategy

El algoritmo de variación de precios —pieza más variable y sensible a la personalización comercial del producto— se modela como una familia de estrategias intercambiables. El comerciante que administra su instancia del software puede seleccionar, mediante configuración, qué estrategia aplicar (decaimiento por

tiempo, apreciación por escasez de stock, volatilidad inversa, u otras futuras) sin requerir un nuevo despliegue de código.

2.4 Patrón Repository

El acceso a los datos persistentes (cupones, historial de cotizaciones, liquidaciones) se abstrae detrás de repositorios. Esto permite que la lógica de negocio se pruebe de forma aislada, sin depender de una base de datos real, y facilita migrar de motor de almacenamiento si un cliente corporativo así lo requiere.

2.5 Arquitectura Dirigida por Eventos (Event-Driven)

Cada transición relevante del ciclo de vida de un cupón —emisión, retención, reventa o expiración— se modela como un evento de dominio. Los módulos secundarios (notificaciones al usuario, generación de reportes contables, alimentación de dashboards) reaccionan a estos eventos de forma asíncrona, sin insertarse en el camino crítico de la transacción principal y sin acoplarse directamente a la lógica del motor de precios.

3. Contextos Acotados del Dominio (Bounded Contexts)

El sistema se organiza en cuatro contextos de dominio independientes, cada uno con entidades, reglas y ciclo de vida propios. Esta división permite que distintos equipos —o distintas fases de un roadmap de implementación— avancen en paralelo sin generar dependencias rígidas entre sí.

Contexto	Responsabilidad Principal	Entidades Clave
Catálogo / Inventario	Administra la definición de productos, servicios o activos que el comerciante ofrece, así como el stock disponible por unidad.	Producto, Unidad de Stock, Categoría
Gestión de Cupones	Gobierna el ciclo de vida del "Cupón de Valor": emisión, ventana de trading, canje, reventa y expiración.	Cupón, Ventana de Trading, Propietario
Motor de Fluctuación de Precios	Calcula y publica el valor de recompra vigente de cada cupón activo, según la estrategia configurada por el comerciante.	Estrategia de Precio, Cotización, Historial de Cotizaciones
Liquidación y Pagos	Ejecuta la contrapartida monetaria de una reventa o un canje: cálculo de comisiones, liquidación neta y registro contable.	Liquidación, Comisión, Movimiento de Fondos

Los cuatro contextos se comunican entre sí exclusivamente a través de eventos de dominio o de los puertos definidos por cada uno; ningún contexto accede directamente a las tablas o estructuras internas de otro. Esto es lo que permite, por ejemplo, sustituir el Motor de Fluctuación de Precios por una versión más sofisticada en el futuro sin reescribir la Gestión de Cupones.

4. Motor de Precios — Diseño Conceptual

El Motor de Precios es el componente de mayor sensibilidad estratégica del sistema, ya que traduce la "Curva de Variación Personalizable" del Documento de Visión en un mecanismo de cálculo operable, auditable y configurable por tenant.

4.1 Responsabilidades del Núcleo

- **Resolver la cotización vigente** de un cupón en cualquier instante, a partir de su valor de emisión, el tiempo transcurrido, el stock remanente y la estrategia activa del comerciante.
- **Recalcular periódicamente** el valor de los cupones activos (mecanismo de "oráculo inverso"), manteniendo actualizado el estado del mercado en tiempo real.
- **Aplicar límites de piso y techo** configurables, para acotar el riesgo tanto del usuario final como del comerciante.
- **Registrar cada cálculo** de forma trazable, de modo que el comerciante y el usuario puedan auditar por qué un cupón vale lo que vale en un momento dado.

4.2 Puertos de Entrada (casos de uso expuestos por el dominio)

Puerto	Responsabilidad	Quién lo invoca
Consulta de Precio	Devuelve la cotización actual de un cupón y su historial en un rango de fechas.	API REST, panel de usuario final
Recálculo de Precio	Dispara el recálculo de un cupón puntual o de todos los cupones activos de un tenant.	Servicio de recálculo periódico (ticker), tareas administrativas
Negociación de Cupón	Ejecuta las dos acciones mutuamente excluyentes del ciclo de vida: canje o reventa.	API REST, app del usuario final

4.3 Puertos de Salida (dependencias que el núcleo requiere del exterior)

Puerto	Propósito
Repositorio de Cupones	Persistir y recuperar el estado de cada cupón (valor emitido, stock, ventana de trading, estado actual).
Repositorio de Historial de Cotizaciones	Registrar cada cálculo de precio realizado, con fines de auditoría y análisis.
Caché de Precios	Servir consultas de lectura de alta frecuencia sin sobrecargar el almacenamiento principal.
Publicador de Eventos	Notificar al resto del sistema cuando cambia el estado de un cupón o se genera una nueva cotización.

Puerto	Propósito
Configuración de Estrategia	Resolver qué estrategia de precio y qué parámetros están activos para un tenant determinado.

4.4 Estrategias de Precio Configurables

El catálogo inicial de estrategias contempla, como mínimo, las siguientes variantes, todas intercambiables sin cambios en el núcleo del motor:

Estrategia	Lógica Conceptual	Caso de Uso Típico
Decaimiento Lineal por Tiempo	El valor de recompra disminuye de forma proporcional a las horas transcurridas desde la emisión, hasta un valor piso.	Alimentos, servicios perecederos, incentivo al consumo inmediato.
Apreciación por Oferta y Demanda	El valor de recompra aumenta a medida que el stock disponible se reduce, reflejando escasez.	Ediciones limitadas, entradas a eventos, productos de alta demanda.
Volatilidad Inversa	Combina ambos factores (tiempo y escasez) con pesos configurables, generando una curva híbrida.	Categorías mixtas o comerciantes que buscan un comportamiento más sofisticado del mercado.

4.5 Adaptadores Concretos Sugeridos

Aunque el núcleo del motor es agnóstico a la tecnología, se recomienda una primera implementación de referencia con el siguiente mapeo de adaptadores, priorizando consistencia transaccional y baja latencia de lectura:

Puerto	Adaptador de Referencia	Justificación
Repositorio de Cupones / Historial	Base de datos relacional (PostgreSQL)	Garantiza consistencia ACID para estados financieros y soporta particionamiento para el alto volumen del historial de cotizaciones.
Caché de Precios	Almacén en memoria (Redis)	Permite servir lecturas de cotización en tiempo real sin saturar la base de datos principal.
Publicador de Eventos	Broker de mensajería (NATS o Kafka)	Desacopla el flujo transaccional principal de los procesos secundarios (notificaciones, reportería).
Capa de Auditoría Inmutable (opcional)	Red compatible con EVM o red empresarial privada	Para clientes que exigen trazabilidad pública o semi-pública de la emisión y reventa de cupones.

La elección de un adaptador de referencia no impide que cada instancia White-Label sustituya cualquiera de estas piezas por la tecnología que prefiera el cliente corporativo, respetando el contrato del puerto correspondiente.

5. Modelo de Datos Conceptual

El modelo de datos se describe aquí a nivel conceptual —entidades, atributos y relaciones— para mantener el documento independiente del motor de base de datos elegido en cada implementación, si bien se recomienda una base relacional por los requisitos de consistencia financiera ya expuestos.

5.1 Entidades Principales

Entidad	Descripción
Tenant (Comerciante)	Representa a cada cliente corporativo que licencia el software; es la unidad de aislamiento multi-tenant.
Estrategia de Precio	Configuración vigente por tenant: qué algoritmo de variación aplica y con qué parámetros.
Cupón	Unidad de valor emitida al comprar; representa la propiedad de un producto, servicio o activo agnóstico.
Historial de Cotizaciones	Registro cronológico e inmutable de cada recálculo de valor realizado sobre un cupón.
Liquidación	Registro de la contrapartida monetaria generada por una reventa: monto bruto, comisión y monto neto.

5.2 Atributos de la Entidad Cupón

Atributo	Descripción
Identificador	Clave única del cupón.
Tenant	Comerciante propietario de la instancia a la que pertenece el cupón.
Propietario (usuario final)	Persona que adquirió el cupón y puede ejercer las acciones del ciclo de vida.
Referencia de producto	Identificador agnóstico del catálogo del comerciante (SKU, código de reserva, lote, etc.).
Valor de emisión	Monto original al momento de la compra.
Valor piso / valor techo	Límites configurables que acotan la variación de precio.
Stock total / stock remanente	Cantidad de unidades disponibles bajo la misma referencia, usada por estrategias de escasez.
Estado	Situación actual: activo, canjeado, revendido o expirado.
Ventana de trading	Fecha y hora en que finaliza la posibilidad de operar el cupón.
Fecha y motivo de cierre	Cuándo y por qué el cupón dejó de estar activo (canje, reventa o expiración).

Por tratarse de un sistema financiero-transaccional, se recomienda que todos los montos se almacenen como valores enteros en la unidad mínima de la moneda (p. ej. centavos), evitando errores de redondeo por punto flotante.

5.3 Atributos del Historial de Cotizaciones

Atributo	Descripción
Cupón asociado	Referencia al cupón cotizado.
Valor cotizado	Resultado del cálculo de la estrategia en ese instante.
Estrategia aplicada	Nombre de la estrategia utilizada para ese cálculo.
Variables de entrada	Snapshot de los datos usados (tiempo transcurrido, stock remanente, parámetros), para auditoría.
Momento de la cotización	Marca de tiempo del recálculo.

Dado el alto volumen de escritura que genera el recálculo periódico, esta entidad debe diseñarse pensando en particionamiento por fecha desde el inicio, para sostener el crecimiento del historial sin degradar el rendimiento de lectura.

5.4 Atributos de Liquidación

Atributo	Descripción
Cupón asociado	Cupón que originó la liquidación.
Tipo de liquidación	Actualmente, únicamente reventa al mercado.
Monto bruto	Valor cotizado en el momento de la reventa, antes de comisión.
Comisión aplicada	Monto retenido por el comerciante según su configuración.
Monto neto	Monto efectivamente transferido al usuario final.

5.5 Relaciones entre Entidades

Cada Tenant puede tener múltiples Cupones y una única Estrategia de Precio activa en un momento dado. Cada Cupón acumula múltiples registros en el Historial de Cotizaciones a lo largo de su ventana de trading, y genera, como máximo, una única Liquidación si su cierre ocurre por reventa. Esta cardinalidad refuerza la regla de negocio de exclusividad mutua entre canje y reventa: un cupón cerrado no puede volver a generar cotizaciones ni liquidaciones adicionales.

5.6 Estrategia de Crecimiento y Retención de Datos

- **Particionamiento temporal:** el Historial de Cotizaciones debe segmentarse en particiones periódicas (p. ej. mensuales), permitiendo archivar o purgar datos antiguos sin afectar el rendimiento de las consultas recientes.
- **Aislamiento por tenant:** todo acceso a datos debe filtrarse siempre por el identificador del tenant, tanto por seguridad como para permitir estrategias de escalado horizontal por cliente en el futuro.
- **Política de retención:** cada instancia White-Label debe poder definir cuánto tiempo conserva el detalle fino del historial de cotizaciones antes de resumirlo o archivarlo, según sus propias obligaciones regulatorias.

6. Arquitectura de la API (REST y Tiempo Real)

La superficie de integración se divide en dos canales complementarios: una API REST para operaciones transaccionales (emitir, canjear, revender, configurar) y un canal de comunicación en tiempo real para la difusión de cambios de precio y eventos del ciclo de vida.

6.1 Principios de Diseño de la API

- **Multi-tenencia transparente:** el tenant se resuelve a partir de la credencial de autenticación de cada llamada, nunca como un parámetro que el cliente pueda manipular libremente.
- **Montos explícitos y sin ambigüedad:** toda cifra monetaria se expresa siempre junto con su moneda, evitando conversiones implícitas.
- **Eventos como ciudadanos de primera clase:** toda transición relevante del ciclo de vida de un cupón se traduce en un evento de dominio consumible tanto por el canal en tiempo real como por integraciones externas del comerciante.
- **Separación entre lectura y escritura:** las consultas de cotización se sirven preferentemente desde la caché, mientras que las operaciones de canje o reventa siempre se validan contra el estado consistente de la base transaccional.

6.2 Catálogo de Endpoints — Catálogo y Emisión

Operación	Propósito
Emisión de cupón	Registra la compra de un producto o servicio y genera el cupón correspondiente, iniciando su ventana de trading.
Consulta de cupón	Recupera el detalle y el estado actual de un cupón puntual.
Listado de cupones por usuario	Recupera todos los cupones activos e históricos asociados a un usuario final.

6.3 Catálogo de Endpoints — Motor de Precios

Operación	Propósito
Consulta de cotización vigente	Devuelve el valor actual de recompra de un cupón.
Consulta de historial de cotizaciones	Devuelve la serie temporal de valores de un cupón, útil para visualizaciones.
Configuración de estrategia de precio	Permite al administrador del tenant seleccionar y parametrizar la estrategia activa.
Recálculo administrativo	Operación interna utilizada por el proceso periódico de recálculo (oráculo inverso).

6.4 Catálogo de Endpoints — Ciclo de Vida del Cupón

Operación	Propósito
Canje de cupón	Ejercicio de retiro inmediato del producto o servicio; cierra el cupón de forma definitiva.
Reventa de cupón	Solicitud de venta del cupón de vuelta al mercado; calcula la liquidación y cierra el cupón.
Consulta de liquidación	Recupera el detalle de una liquidación ya ejecutada (monto bruto, comisión, monto neto).

6.5 Catálogo de Endpoints — Administración (White-Label)

Operación	Propósito
Panel de métricas del tenant	Expone indicadores clave: proporción de canje inmediato vs. especulación, ingresos por comisión.
Configuración de límites anti-fraude	Permite definir topes de volumen o frecuencia de operación por usuario final.

6.6 Canal de Tiempo Real

El canal en tiempo real permite que el usuario final observe la evolución del valor de su cupón sin necesidad de refrescar manualmente. Cada cliente se suscribe a un canal específico por cupón o por tenant, y recibe los siguientes tipos de evento:

- Actualización de cotización: se emite cada vez que el motor de precios recalcula el valor de un cupón activo.
- Cupón emitido: se emite al completarse una compra.
- Cupón canjeado: se emite cuando el usuario ejerce el retiro inmediato.
- Cupón revendido: se emite cuando el usuario ejerce la reventa al mercado.
- Cupón expirado: se emite cuando la ventana de trading concluye sin acción del usuario.

6.7 Flujo de Interacción de Extremo a Extremo

El recorrido típico de un cupón, desde la perspectiva de la API, sigue esta secuencia:

- **1.** El usuario final compra un producto y el sistema emite el cupón correspondiente, iniciando la ventana de trading.
- **2.** El proceso periódico de recálculo evalúa el cupón según la estrategia activa del tenant y publica una nueva cotización, tanto en la caché de lectura como en el historial persistente.
- **3.** El canal en tiempo real notifica al usuario final el nuevo valor, permitiéndole decidir con información actualizada.

- **4.** El usuario ejerce una de las dos acciones excluyentes: canje inmediato o reventa al mercado.
- **5.** Si la acción fue una reventa, el sistema calcula la liquidación (monto bruto, comisión y monto neto) y notifica el cierre del cupón a través de un evento de dominio.

7. Requisitos No Funcionales

7.1 Escalabilidad y Concurrencia

El sistema debe soportar picos de lectura intensivos sobre las cotizaciones (múltiples usuarios observando el mismo cupón o el mismo catálogo en simultáneo) sin que ello degrade la velocidad de las operaciones de escritura (canje, reventa, emisión). La separación entre una caché de lectura y una base transaccional de escritura, descrita en la sección 4.5, es la respuesta arquitectónica a este requisito.

Adicionalmente, dado que dos usuarios podrían intentar operar sobre unidades del mismo lote de stock en el mismo instante, el sistema debe garantizar control de concurrencia a nivel de cada cupón individual, de forma que nunca se permita un doble canje, una doble reventa, o una condición de carrera entre ambas acciones.

7.2 Seguridad

- **Autenticación y autorización:** toda operación debe estar asociada de forma inequívoca a un tenant y, dentro de él, a un usuario o rol administrativo autorizado.
- **Aislamiento multi-tenant:** los datos de un comerciante nunca deben ser accesibles, ni siquiera de forma indirecta, por otro comerciante que use la misma instancia del software.
- **Cumplimiento normativo:** los flujos que impliquen devolución de dinero al usuario final deben poder integrarse con procesos de verificación de identidad y prevención de lavado de activos (KYC/AML), de forma configurable según el volumen y la naturaleza de cada comerciante.
- **Protección contra manipulación de mercado:** deben existir controles que impidan que un mismo usuario, o un grupo coordinado de usuarios, opere de forma repetitiva para explotar artificialmente el algoritmo de precios.

7.3 Observabilidad

Cada componente del sistema —en particular el motor de precios y el módulo de liquidaciones— debe emitir métricas operativas (latencia de cálculo, volumen de recálculos, tasa de errores) y registros de auditoría suficientes para reconstruir, ante cualquier disputa, la secuencia exacta de eventos que llevó a un cupón a su estado final. Esta trazabilidad no es opcional: es un requisito directo de la filosofía de transparencia declarada en el Documento de Visión.

7.4 Resiliencia y Tolerancia a Fallos

El proceso periódico de recálculo de precios (oráculo inverso) debe diseñarse asumiendo que puede fallar parcialmente sin comprometer la integridad de los cupones: un fallo en el cálculo de un cupón no debe bloquear el recálculo del resto del lote. De la misma manera, la publicación de eventos hacia módulos secundarios (notificaciones, reportería) debe estar desacoplada del camino crítico transaccional, de forma que un problema en esos módulos nunca impida completar un canje o una reventa.

7.5 Estrategia de Pruebas

- **Pruebas de dominio aisladas:** la lógica de cálculo de precios y las reglas del ciclo de vida del cupón deben poder validarse sin depender de una base de datos real, gracias al desacoplamiento que ofrecen los puertos y adaptadores.
- **Pruebas de integración por adaptador:** cada adaptador concreto (repositorio, caché, publicador de eventos) debe contar con pruebas específicas contra la tecnología real que implementa.
- **Pruebas de extremo a extremo del ciclo de vida:** escenarios completos de emisión, decisión y cierre de un cupón, incluyendo los casos límite de expiración y de concurrencia entre canje y reventa.

8. Infraestructura y Consideraciones de Despliegue

8.1 Empaquetado y Portabilidad

Dado que el producto se comercializa como software White-Label, cada componente (motor de precios, gestión de cupones, liquidaciones, worker de recálculo) debe poder empaquetarse y desplegarse de forma independiente, permitiendo que distintos clientes corporativos elijan su propia infraestructura de nube o su propio centro de datos on-premise sin comprometer la arquitectura interna.

8.2 Entornos y Ciclo de Publicación

Se recomienda mantener al menos tres niveles de entorno —desarrollo, validación (staging) y producción— con un proceso de integración y entrega continua que permita desplegar nuevas estrategias de precio, adaptadores o mejoras del núcleo sin interrumpir las ventanas de trading activas de los cupones ya emitidos.

8.3 Aislamiento de Infraestructura por Tenant

Para clientes corporativos con requisitos de volumen o de cumplimiento normativo particularmente exigentes, la arquitectura debe permitir escalar desde un modelo de infraestructura compartida (multi-tenant lógico) hacia un modelo de infraestructura dedicada por tenant, sin necesidad de rediseñar el núcleo de dominio.

9. Consideraciones de Seguridad y Cumplimiento Normativo

Tal como se anticipó en el Documento de Visión, el modelo de negocio de SMCTA involucra la recompra monetaria de un instrumento emitido por el comercio, lo cual puede asemejarse, según la jurisdicción, a instrumentos financieros o mecanismos regulados de dinero electrónico. Desde el punto de vista técnico, esto se traduce en los siguientes requisitos de diseño:

- **Trazabilidad completa:** cada cotización, canje, reventa y liquidación debe quedar registrado de forma inmutable y consultable, incluso años después de ocurrido.
- **Puntos de extensión para KYC/AML:** el flujo de reventa debe contemplar un punto de extensión donde insertar verificaciones adicionales de identidad o de origen de fondos, activable por configuración según el perfil regulatorio de cada tenant.
- **Exposición clara del riesgo al usuario final:** la API y las interfaces que consuman sus datos deben poder comunicar, en cada cotización, que el valor de recompra no está garantizado y puede devaluarse.

10. Roadmap Técnico y Próximos Pasos

- **Arquitectura de Decisión (ADRs):** documentar formalmente las decisiones clave —lenguaje de backend, motor de base de datos, broker de eventos— con sus alternativas evaluadas y su justificación.
- **Prueba de concepto del Motor de Precios:** validar con datos reales o simulados el comportamiento de al menos dos estrategias de precio bajo carga concurrente.
- **Definición del modelo de comisiones:** formalizar las reglas de negocio de la sección de Reglas de Negocio de Alto Nivel del Documento de Visión en parámetros configurables concretos.
- **Especificación de contratos de API:** producir el contrato formal (esquema de solicitud/respuesta) de cada endpoint descrito en la sección 6, como insumo directo para el equipo de implementación.
- **Revisión legal y regulatoria por mercado:** confirmar, junto con asesoría legal especializada, los puntos de extensión de cumplimiento normativo antes de la primera comercialización White-Label.

Este documento técnico complementa el Documento de Visión de SMCTA y constituye la base arquitectónica de referencia para la fase de implementación; las decisiones de bajo nivel (lenguaje de programación, esquema físico de base de datos, contratos de API en detalle) se desarrollarán en documentos técnicos derivados.